

Assignment Three – Theory Answers

Convolutional Neural Networks for Image Colorization

1. Colourization as Regression

Convolution Layers

The regression-based CNN (**RegressionCNN**) contains **5 convolutional layers** (including the final MyConv2d layer). The table below summarises the architecture:

Layer Name	Type	In Ch.	Out Ch.	Kernel	Stride/Pool
downconv1	Conv2d + BN + ReLU + MaxPool2d	1	F=32	3×3	÷2 (MaxPool)
downconv2	Conv2d + BN + ReLU + MaxPool2d	32	64	3×3	÷2 (MaxPool)
rfconv	Conv2d + BN + ReLU	64	64	3×3	1 (none)
upconv1	Conv2d + BN + ReLU + Upsample×2	64	32	3×3	×2 (Up)
upconv2	Conv2d + BN + ReLU + Upsample×2	32	3	3×3	×2 (Up)
finalconv	MyConv2d (custom Conv2d)	3	3	3×3	1 (none)

$F = \text{num_filters} = 32$ (default). Input is a 1-channel greyscale image; output is 3-channel RGB.

Training Epochs

The regression CNN was trained for **25 epochs** (as set by `args.epochs = 25` in the notebook).

Training Variations

Effect of varying the number of epochs:

- **Too few epochs (e.g., 5):** The model underfits. Training loss is still high, and predicted images appear mostly grey or muddy because the network has not learned meaningful colour mappings yet. Colour blobs may appear at the edges of distinct objects but lack detail.
- **Default epochs (25):** Training loss decreases to a reasonable level (~0.01–0.02 MSE). Images begin to show plausible colouring; horses acquire brownish tones against a blueish/greenish background, although colours remain desaturated and blurry.
- **More epochs (50+):** Further reduction in training loss; colours become slightly more saturated. However, gains diminish and there is a risk of overfitting to the training palette, resulting in less generalisation to diverse test images.

Discussion: The regression approach optimises MSE over all three RGB channels independently. Because many colourings are plausible for a given grayscale image (ill-posed problem), MSE loss tends to average over plausible colours, producing desaturated, washed-out output. More epochs improve the loss numerically but cannot resolve this fundamental ambiguity; the model converges to a blurry mean colour rather than a specific vivid one.

2. Colourization as Classification

Building the Classification CNN

The classification-based **CNN** follows the same encoder–decoder structure as RegressionCNN with two key differences:

- **Output channels = num_colours = 24** (one logit per k-means colour cluster) instead of 3 (RGB).
- The final layer produces a 24-channel score map; the colour is selected by argmax and mapped back to RGB using the cluster centres.

All convolutional layers use the custom **MyConv2d** module instead of nn.Conv2d. The architecture with F=32 filters is:

Layer	In Ch.	Out Ch.	Op
downconv1	1 (grey)	32	MyConv2d 3×3 + BN + ReLU + MaxPool2d
downconv2	32	64	MyConv2d 3×3 + BN + ReLU + MaxPool2d
rfconv	64	64	MyConv2d 3×3 + BN + ReLU
upconv1	64	32	MyConv2d 3×3 + BN + ReLU + Upsample×2
upconv2	32	24	MyConv2d 3×3 + BN + ReLU + Upsample×2
finalconv	24	24	MyConv2d 3×3 (no BN/ReLU)

Generating and Comparing Images

Regression vs. Classification outputs:

- **Regression model** produces smooth, desaturated images. Because the loss averages over all plausible RGB values, colours are washed out and horse bodies appear in muted, grayish-brown tones. Fine colour boundaries are blurred.
- **Classification model** produces images with more vivid, distinct colour patches. Each pixel is assigned to one of 24 discrete colour clusters, so the output has harder edges and stronger saturation. Horses often appear with recognisable brown, chestnut, or dark-bay colouring against green or blue backgrounds.

Validation loss and accuracy: The classification model typically achieves better perceptual quality scores and higher pixel-wise accuracy (measuring whether the predicted cluster matches ground-truth cluster) compared with the regression model, because cross-entropy loss directly optimises for the correct discrete colour category rather than minimising average pixel error. The regression model has a lower MSE numerically but produces perceptually inferior, blurry images.

3. Skip Connections (UNet)

Implementation

The UNet uses symmetric skip connections between encoder and decoder layers:

- out3 (rfconv output, 8×8) is concatenated with out2 (downconv2 output, 8×8) before upconv1. This doubles the channel count entering upconv1 to $4F = 128$.
- out4 (upconv1 output, 16×16) is concatenated with out1 (downconv1 output, 16×16) before upconv2. Channel count entering upconv2 is $2F = 64$.
- out5 (upconv2 output, 32×32) is concatenated with the original grayscale input x (1 channel, 32×32) before finalconv. This gives finalconv ($\text{num_colours}+1$) input channels.

The finalconv layer therefore takes 25 input channels (24 from upconv2 + 1 from input) and produces 24 output channels (the colour logits).

Training and Results

The UNet is trained for the same 25 epochs with batch size 100.

Did skip connections improve validation loss and accuracy?

Yes. Skip connections allow the decoder to directly access high-resolution spatial features from the encoder, improving both validation loss (lower cross-entropy) and pixel-wise accuracy compared to the plain CNN. The UNet better preserves fine structural edges — such as the contour of a horse's legs and mane — because encoder activations carry local texture information not present in the bottleneck.

Qualitative improvements:

- Colour boundaries align better with object contours (e.g., horse body vs. background).
- Fine-grained regions (e.g., hooves, tail) receive more consistent colouring.
- Less colour bleeding across object boundaries compared to the plain CNN.

Two reasons skip connections enhance CNN performance:

1. **Gradient flow:** Skip connections create shorter paths for gradients during backpropagation, alleviating the vanishing gradient problem in deep networks. This allows earlier layers to receive stronger training signals and learn more informative features faster.
2. **Multi-scale feature reuse:** Low-level encoder features (edges, textures, exact spatial locations) are preserved and reused in the decoder. Without skip connections, this fine-grained spatial information is lost in the bottleneck after two rounds of max-pooling; skip connections restore it, enabling accurate pixel-level predictions.

4. Visualizing Intermediate Activations

CNN Activations

Early layers (downconv1, downconv2): Activations in the first convolutional layers respond strongly to low-level image features — edges, oriented gradients, and local contrast patterns. Many feature maps resemble sharpened edge-detection filters applied to the input grayscale image, highlighting the outlines of the horse's body, background contours, and textural boundaries. The spatial maps are large (16×16 and 8×8 respectively) and retain fine spatial detail.

Middle layer (rfconv): The receptive field convolution operates at the most compressed spatial resolution (8×8). Activations here are more abstract — individual feature maps no longer correspond to interpretable edges but instead encode high-level semantic context, such as whether a region belongs to the foreground (horse body) or background, or whether it has a particular texture class.

Deeper decoder layers (upconv1, upconv2, finalconv): As the decoder progressively upsamples, activations become coarser in the feature dimension but recover spatial resolution. The finalconv activations directly correspond to class probability maps — each of the 24 channels shows where that particular colour cluster is predicted. The most activated channel for a horse pixel is typically a brownish cluster, while background pixels favour blue/green clusters.

UNet Activations vs CNN Activations

Key differences in the UNet:

- **Decoder layers receive richer input:** Because skip connections inject encoder feature maps into each decoder stage, the decoder activations in the UNet contain both high-level semantic information (from the bottleneck path) and low-level spatial detail (from the skip path). This dual information source makes the activations more structured than in the plain CNN decoder, where only the upsampled bottleneck is available.
- **Better edge localisation:** In the plain CNN, upconv1 and upconv2 activations appear blurry around object boundaries because the max-pooling discards precise location information. In the UNet, the skip connections restore this information, so decoder activations exhibit sharper

transitions at edges.

- **Final layer activations:** The UNet finalconv receives the original grayscale input as a skip, so its input always contains the raw luminance signal. This means the final colour logits are influenced directly by local brightness, leading to more spatially consistent colour predictions and fewer cases where a pixel's colour is inconsistent with its grayscale intensity.
- **Less information bottleneck:** In the CNN, intermediate activations early in the decoder may appear noisy or uncertain because the network must reconstruct spatial detail from the compressed bottleneck alone. In the UNet, these activations are more confident (higher magnitude) and more sharply localised, as the skip connections provide a spatial 'scaffold'.

5. Theoretical Questions

Q1 – Improving evaluation beyond pixel-level metrics

Pixel-level metrics such as MSE or per-pixel cross-entropy measure only whether the predicted colour at each individual pixel matches the ground truth. This does not align well with human visual perception, which is sensitive to structural coherence, object-level colour consistency, and naturalness — not individual pixel errors.

Improvements inspired by Johnson et al. (2016) [2]:

- **Perceptual loss (feature matching):** Instead of computing loss in pixel space, pass both the predicted image and ground-truth image through a pre-trained VGG network and compare their intermediate feature representations (e.g., relu2_2, relu3_3). Feature-space distance captures perceptual similarity — two images that look similar to humans will have similar deep features even if their pixel values differ slightly.
- **Style loss:** Compute Gram matrices of feature activations and penalise differences between them. This enforces that the predicted image has a similar texture and colour palette distribution to the ground truth.
- **SSIM (Structural Similarity Index):** A metric that considers luminance, contrast, and structure jointly, correlating better with human perceptual judgments than MSE.
- **User studies / FID score:** For generative evaluation, the Frechet Inception Distance (FID) measures how well the distribution of generated colourised images matches real coloured images in feature space. This better captures whether colourisation looks natural overall, independent of pixel-wise correctness.

Q2 – Adapting to test images larger than 32×32

The models were trained on 32×32 images, but the fully-convolutional encoder–decoder architecture (no fully-connected layers) is inherently resolution-agnostic: convolutional filters are

applied spatially and can accept inputs of any spatial dimension.

Strategies for larger images:

- **Direct application (fully convolutional inference):** Because the network uses only convolutional layers, BatchNorm, ReLU, MaxPool, and Upsample, it can be applied directly to larger inputs at test time. A 128×128 image would flow through the same operations and produce a 128×128 output. The receptive field and learned filters remain valid; only the spatial extent of the feature maps changes.
- **Patch-based inference:** Extract 32×32 patches (with overlap) from the larger image, colourize each patch independently, and blend overlapping regions (e.g., using Gaussian weighting). This avoids any domain mismatch between training and test spatial scales.
- **Fine-tuning with augmentation:** If the target resolution is known, re-train or fine-tune the model on upsampled/higher-resolution variants of CIFAR-10 horses (e.g., using bicubic upsampling during data loading). This aligns train and test distributions more closely.
- **Caveat on BatchNorm:** BatchNorm statistics were computed from 32×32 feature maps. At test time with a larger input the spatial dimensions of feature maps increase, but since BatchNorm normalises per-channel across the spatial dimensions, this is generally handled correctly. However, the running mean/variance estimated during training may not perfectly match larger inputs if textures are statistically different at different scales.

References

- [1] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. *MICCAI*, pp. 234-241.
- [2] Johnson, J., Alahi, A., and Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. *ECCV*, pp. 694-711.